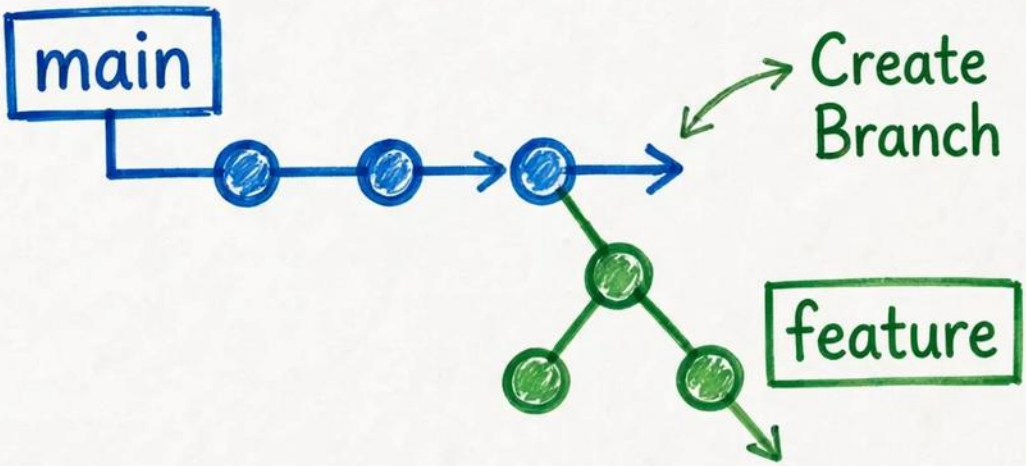
30 GIT INTERVIEW SCENARIOS

Production Edition Every Developer Should Know



Make Changes → Commit → Merge

- 30 Most Important Production Git Scenarios
- Clear Explanation + Diagrams
- Perfect for Interviews & Real-world

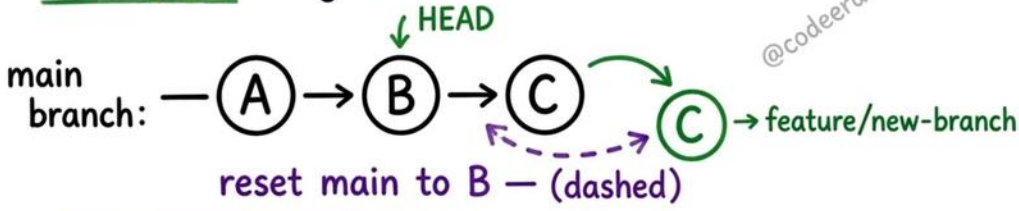
① SCENARIO 1

SCENARIO:
Committed to wrong branch in production, how to move it to correct branch?

ANSWER:

- git branch new-branch — moves commit, or
  git checkout -b new-branch then reset old branch

EXAMPLE diagram:



@codeera.tech

KEY DIFFERENCE:
Safe move — the new branch now holds the commit, so you can safely reset the old branch back without losing any history.

---

② SCENARIO 2

SCENARIO: You pushed .env secrets to main, how to remove safely?

ANSWER: Use git filter-repo / BFG Repo-Cleaner — rewrites history.

SOLUTION steps:

① Back up repo:
git clone --mirror <url>

SOLUTION:

② Remove file:
git filter-repo --force --invert-paths --path .env

③ Force push:
git push --force --all &&
git push --force --tags

④ Rotate secrets immediately.

NOTE: Force-push rewrites history coordinate with your team!

3

SCENARIO: Production is down, need to rollback last deploy quickly, how?

ANSWER:

Use git revert not reset on shared main.

SOLUTION steps:

1) git log --oneline to find bad commit
2) $ git revert <bad-commit-hash>
3) $ git push origin main
4) For multiple $ git revert HEAD~3..HEAD

EXAMPLE diagram:

Before: Ⓐ → Ⓑ → Ⓒ → Ⓓ★
(D marked as bad/red)

After: Ⓐ → Ⓑ → Ⓒ → Ⓓ★ → Ⓡ ✓
(R is green revert commit)

TIP revert is safe for shared branches creates new commit.

⚠ NOTE: Never force push rollback on main.

4

SCENARIO: 4 What is difference between git reset and git revert for production hotfix?

KEY DIFFERENCE	git reset	git revert	
Creates new commit	No	Yes	
History	Rewrites	Preserved	
Safe on shared	No	Yes	
Use	Local	Production	

git reset:

HEAD ← moves>C HEAD ⟷ Ⓐ → Ⓑ → Ⓒ
rewinds history

git revert:

Ⓐ → Ⓑ → Ⓒ → Ⓓ → Ⓡ → R
adds new commit R forward

5 SCENARIO 5

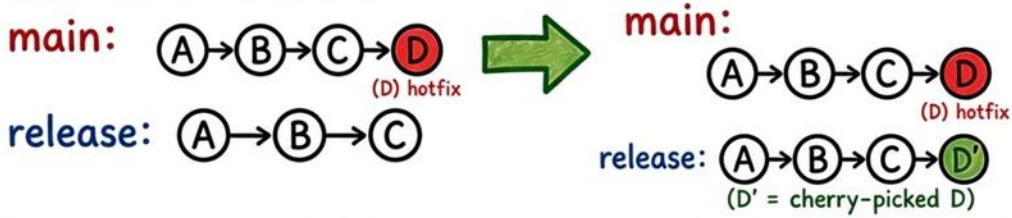
SCENARIO: How to cherry-pick critical hotfix from main to release branch?

ANSWER: git cherry-pick moves single commit to to another branch.

STEPS:

1) $ git checkout release
2) $ git cherry-pick <commit-hash>
3) Resolve conflicts if any
4) $ git push origin release

EXAMPLE diagram:



6 SCENARIO 6

TIP: Use -x to keep reference reference to original commit.

SCENARIO: How to squash multiple commits into one before PR?

ANSWER: Squashing combines multiple commits into one for clean history.

STEPS:

1) $ git rebase -i HEAD~4
2) keep first as pick, others as squash
3) Save & exit editor.

EXAMPLE: feature branch — BEFORE:

WHEN TO USE:

- ✓ clean history before PR
- ✓ combine fixup/WIP commits

(D' = B + C + D like)

NOTE: Don't squash on shared main! Use --force-with-lease after rebase.

7

SCENARIO:
What is the difference between git merge and git rebase ?

ANSWER:

Both integrate changes from one branch into another, but in different ways.

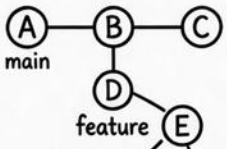
- merge creates a new commit.
- rebase rewrites history by placing your commits on top of the target branch.

KEY DIFFERENCE

Aspect	git merge	git rebase
Creates new commit	Yes	No
History	Preserved (with merge commit)	Rewritten (linear)
Safe on shared branch	Yes	No (avoid)
Best used when	Collaborating	Keeping history clean
Result	Non-linear history	Linear history

EXAMPLE: git merge

Before merge:



After merge:

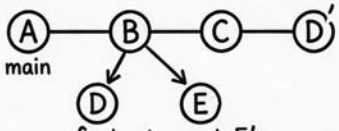


M = merge commit

★ Merge keeps all history and creates a merge commit.

EXAMPLE: git rebase

Before rebase:



feat ..rcture at E'

After rebase (feature on top of main):

D are new commits

★

★ Rebase rewrites history and keeps it linear.

TIP:

- Use merge for team collaboration.
- Use rebase for clean, readable history (before pushing).

8

SCENARIO:
How to resolve merge conflicts ?

ANSWER:

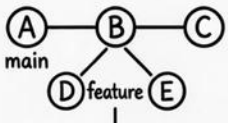
When two branches modify the same line, Git cannot decide automatically. You must resolve it manually.

STEPS TO RESOLVE:

1) Run merge — in small feature
2) Git shows conflicts in files.
3) Open the file and edit the conflict.
   <<<<<<< HEAD
4) $ git add ...
5) $ git commit

Diagram:

BEFORE MERGE:



AFTER MERGE (conflict resolved):

A — B — B — C — M

with D-E merged into M

@codeera.tech

9 SCENARIO 9

SCENARIO:
How to use git stash and git stash pop?

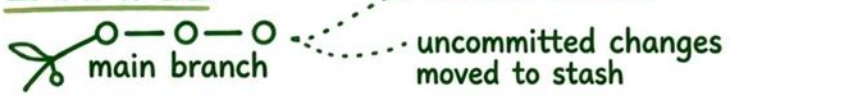
ANSWER:

Stash temporarily saves uncommitted changes so you can switch branches.

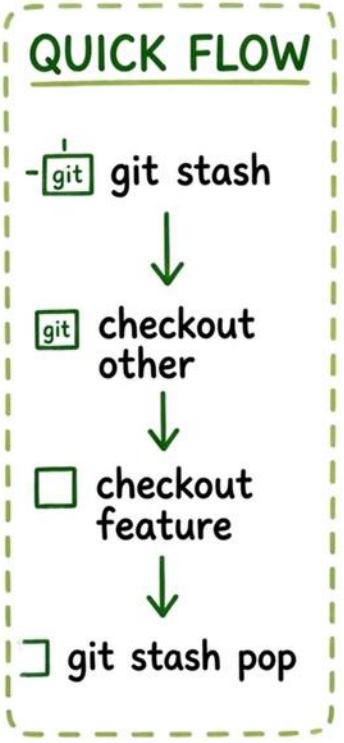
SOLUTION steps

1) $ git stash save
2) $ git checkout main work else
3) $ git checkout feature
4) $ git stash pop apply and remove
5) $ git stash list optional

EXAMPLE



QUICK FLOW



---

10 SCENARIO 10

How to recover a deleted commit using git reflog?

ANSWER: Even if commit seems lost reflog tracks HEAD.

SOLUTION:
1) $ git reflog
2) Find commit hash of lost E
3) $ git checkout safe checkout

EXAMPLE: Initial History

HEAD@{0}	Action	Ref.
HEAD@{1}	commit E	hash...



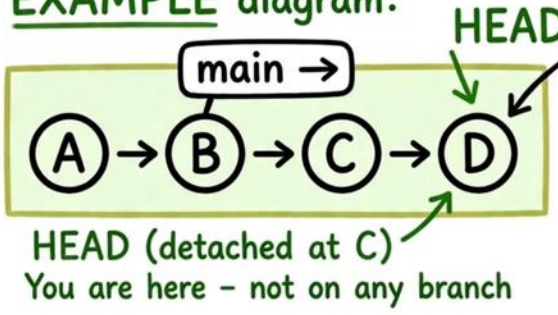
⚠ NOTE: reflog expires! Usually 30-90 days. Recover ASAP.

11 SCENARIO 11

SCENARIO: What is detached HEAD and how do you get out of it?

ANSWER: Detached HEAD means HEAD is pointing directly to a commit — not to a branch, not on any branch, commits can be lost unless you create a branch.

EXAMPLE diagram:



HOW TO GET OUT:

1) `$ git switch -c my-branch`
OR `$ git checkout -b my-branch`
2) `$ git switch main` my-branch → C
→ HEAD now at D (on main)
3) Create branch and continue

QUICK RECAP

- HEAD = direct
- Not on branch
- Make branch to save

---

12 SCENARIO 12

SCENARIO: What is difference between git push --force and --force-with-lease?

ANSWER: Both rewrite history on the remote. --force is dangerous, --force-with-lease is safer.

1) git push --force

What happens?
Forces and overwrites no matter what — can delete others' work.



2) git push --force-with-lease

What happens?
Pushes only if remote is still at the commit you expected; if someone else pushed → it is rejected.



TIP: Use --force-with-lease to avoid accidentally overwriting teammates' work.

NOTE: Force push can overwrite shared history — coordinate with team!

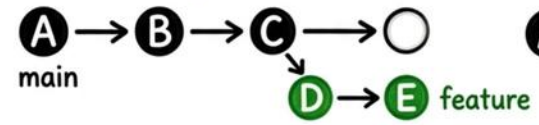
13 SCENARIO 13

SCENARIO: How to recover a deleted branch?

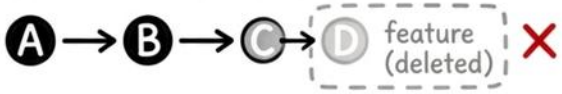
ANSWER: Even if you delete a branch, Git keeps a reference in reflog for some time — you can recover!

EXAMPLE

Before deleting branch:



After Delete branch feature:



SOLUTION steps:

1) $ git reflog
   - shows all actions
   abcd123 HEAD@t0): checkout: moving from feature to main
2) Find commit hash of last commit on feature branch
3) $ git branch feature efgh456
   (use latest hash)
4) $ git checkout feature

AFTER RECOVERY



Branch recovered ✓

14 SCENARIO 14

SCENARIO: How to rename a branch safely?

ANSWER: Use git branch -m to rename locally, then update remote as well.

EXAMPLE before/after:

Before rename:



feature-old

After rename:



feature-new

STEPS:

1) $ git branch -m feature-old feature-new
   or: $ git branch -m feature-new (if on that branch)
2) $ git push origin --delete feature-old



TIP: TIP: reflog is your best friend! Refs kept 90 days then garbage collected.

⚠ NOTE: If reflog entry is too old and removed, recovery not possible!

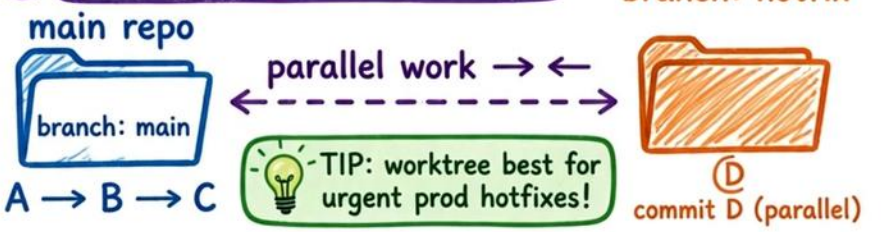
15 SCENARIO 15

SCENARIO: How to work on prod hotfix and feature in parallel without stashing?

ANSWER: Use git worktree — multiple working trees.

SOLUTION:

1. git worktree list
2. git worktree add ../hotfix main
3. cd ../hotfix fix bug
4. git worktree remove ../hotfix



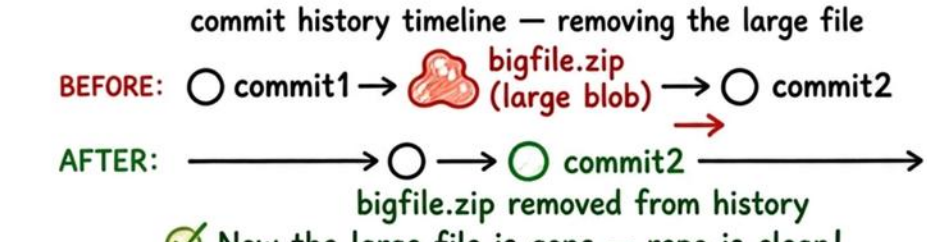
16 SCENARIO 16

SCENARIO: Accidentally committed large file to repo how to remove?

ANSWER: History contains big file even if deleted, need to filter history.

1. git log --stat -- bigfile.zip
2. git filter-repo --invert-paths --path bigfile.zip OR BFG --strip-blobs-bigger-than 50M
3. git push --force --all --tags

⚠ NOTE: Warn team! Force-push rewrites history. Add to .gitignore, use Git LFS for large files!



✓ Now the large file is gone — repo is clean!

17 17 SCENARIO 17

SCENARIO:
How to fix last commit message pushed to main?

ANSWER: Use amend for local but pushed needs care.

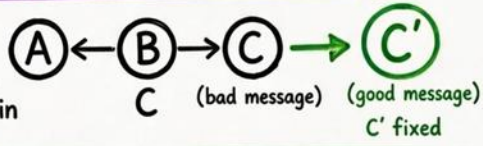
SOLUTION:

1) git commit --amend -m 'fix: correct message'

2) git push --force-with-lease origin main
   * only if team allows

HEAD → C'

Alternative: git revert then new commit with correct message - safer for shared main



TIP: For shared main prefer revert to avoid rewriting history.

18 SCENARIO 18

SCENARIO: How to undo last commit but keep changes for rework?

ANSWER: Use soft reset

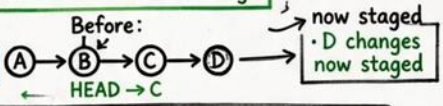
KEY: --soft keeps staged,
--mixed keeps unstaged (default)
--hard deletes everything

SOLUTION:

1) git reset --soft HEAD~1
   - keeps changes staged

2) Edit files

3) git commit -m 'new message' → D changes



Mode	I Keeps	Use case
Soft	- Staged	- Rework
Mixed	- Unstaged	- Unstage
Hard	- Deletes	- Discard

19 SCENARIO 19

SCENARIO: How to sync forked repo with upstream in production setup?

ANSWER: Add upstream remote.

SOLUTION:

1) git remote add upstream original-url
2) git fetch upstream
3) git checkout main
4) git merge upstream/main or rebase
5) git push origin main

Diagram:

upstream:



origin fork (before):



(behind — missing C, D)



after merge:

now synced ✓

TIP: Fetch regularly to avoid large conflicts.

---

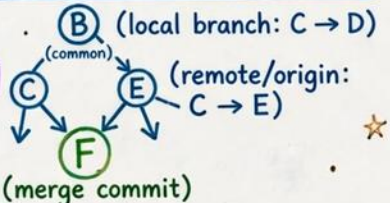
20 SCENARIO 20

SCENARIO: How to resolve divergent branches after force push?

ANSWER: Divergent local and remote have different histories.

SOLUTION:

1) git fetch origin
2) git log --graph --all shows divergence
3) Choose: git merge origin/main or origin/main
4) Resolve conflicts
5) git push



after merge:
F merges the two histories → now synced

Diagram:

NOTE: Avoid force push on shared branches. Use --force-with-lease if needed.

21 SCENARIO 21

SCENARIO: How to tag and version production releases?

ANSWER: Use annotated tags for releases.

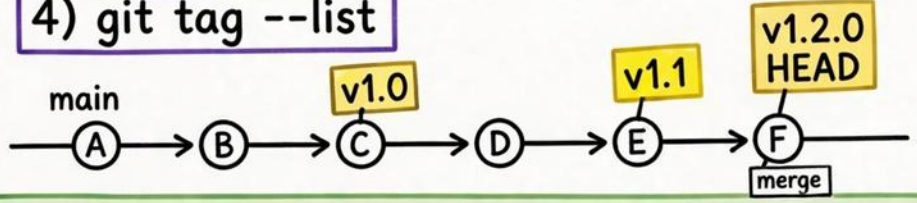
SOLUTION:

1) git tag -a v1.2.0 -m "Release v1.2.0 prod"

2) git push origin v1.2.0 or --tags

3) git checkout v1.2.0 to deploy

4) git tag --list



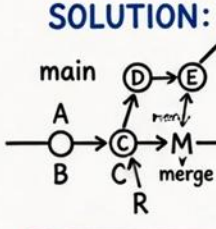
TIP: Semver major.minor.patch - Use annotated tags, not lightweight.

22 SCENARIO 22

SCENARIO: How to revert a merge commit that broke production?

ANSWER: Merge commit revert needs -m flag.

SOLUTION:



1) git log --merges find merge commit M

2) git revert -m 1 <merge-hash> where -m 1 keeps mainline parent

3) git push origin main

after revert R undoes merge: A - B - C - M - R

NOTE: Later to re-merge feature need to revert the revert first!

23 SCENARIO 23

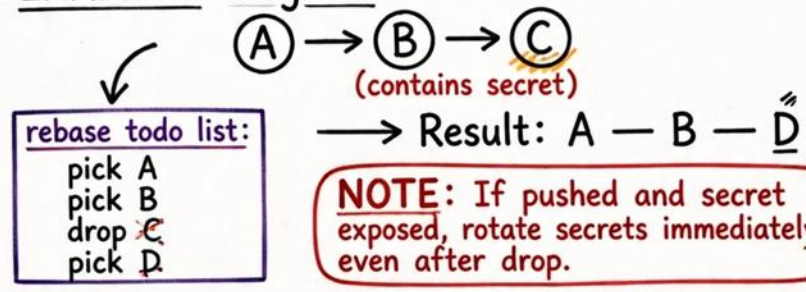
SCENARIO: How to drop a commit containing secrets using interactive rebase?

ANSWER: Use rebase -i to drop.

SOLUTION steps:

1) git rebase -i HEAD~4 - list shows pick commits
2) Change pick to drop for secret commit or delete line
3) Save and close editor
4) git push --force-with-lease if needed

EXAMPLE diagram:



NOTE: If pushed and secret exposed, rotate secrets immediately even after drop.

---

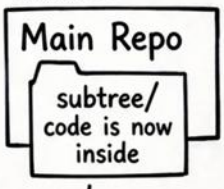
24 SCENARIO 24

SCENARIO: Git submodules vs subtree for prod dependencies?

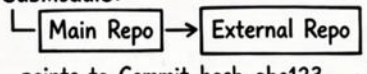
	Submodule	Subtree
Concept:	points to specific commit	merges code into repo
History:	Separate	Unified
Complexity:	Harder	Easier
Update:	git sumodume update	git subtre pull --prefix=path remote branch
Best for:	Large independent libs	Small tightly coupled libs

KEY DIFFERENCE

Subtree:



Submodule:



TIP: Use subtree for simpler workflow if team unfamiliar with submodules.

25 SCENARIO 25

SCENARIO: How to prevent direct push to main using git hooks?

ANSWER: Use pre-push or pre-commit hook + server branch protection.

SOLUTION:

- Local hook: 1) .git/hooks/pre-push file — check if branch == main then exit 1; echo "blocked"
2) chood +x hook. Better Server side: GitHub branch protection rule — Require PR — block direct push

EXAMPLE code (git hook):

```
if [ "$branch" = "main" ]; then
  echo "Direct push to main blocked!"
  exit 1
fi
```

TIP: Combine local + remote protection.

---

26 SCENARIO 26

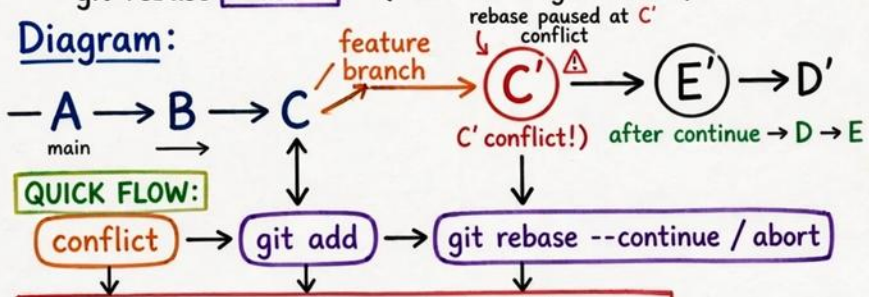
SCENARIO: Rebase failed in middle of release — how to recover?

ANSWER: Rebase can stop on conflict.

SOLUTION steps:

1) Resolve conflict files,
2) git add <file>,
3) • git rebase `--continue` (skip the commit)
   • git rebase `--abort` (return to original state)

Diagram:



NOTE: NOTE: Never force push shared branches.

27 SCENARIO 27

SCENARIO:
How to handle cherry-pick conflict?

ANSWER: Cherry-pick can conflict like merge.

SOLUTION:

1) git cherry-pick `<hash>`
2) CONFLICT shows
3) Edit files — remove markers `<<<< === >>>>`
4) `git add <file>`
5) git cherry-pick --continue or --abort to cancel —

`git cherry-pick --continue` `git chery--abort` `/--skip`



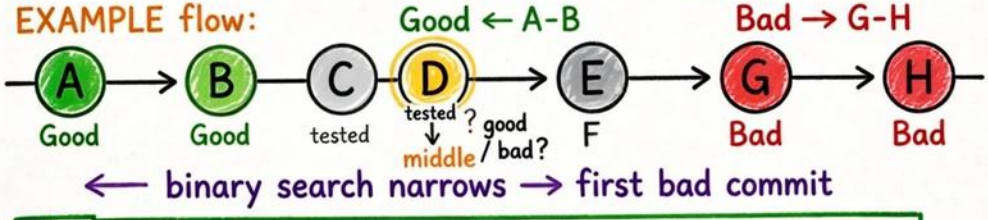
28 SCENARIO 28

SCENARIO: How to find which commit broke production using git bisect?

ANSWER: Bisect uses binary search to find bad commit.

SOLUTION:

1) git bisect `start`
2) git bisect `bad HEAD`
3) git bisect `good <good-hash>`
4) Git checks out middle commit — test and mark: `git bisect` good/bad
5) Repeat until first bad found 6) `git bisect reset`



TIP: Automate with git bisect run npm test.

NOTE: Bisect requires clean working tree.

29 SCENARIO 29

SCENARIO: How to handle line endings and .gitattributes for prod build?

ANSWER: Different OS use LF vs CRLF causes conflicts.

SOLUTION steps:

1) Create .gitattributes file
   * text=auto
   *.sh text eol=lf
   *.bat eol=crlf
2) " git add .gitattributes
3) " git commit

EXAMPLE:

Setting	Result
*.js	text=auto	normalized
*.sh	lf	always lf
*.bat	eol=crlf	always crlf

NOTE: Prevents builds failing on different OS!

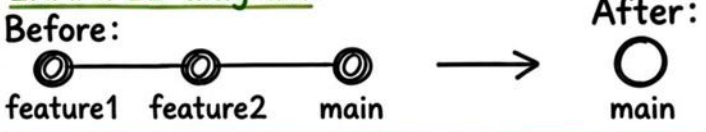
30 SCENARIO 30

SCENARIO: How to clean up local and remote branches after production release?

ANSWER: Keep repo clean, avoid clutter, improve performance.

1. git branch -d feature -D force if unmerged
2. git push origin --delete feature deletes remote
   git push origin --delete feature
3. git fetch --prune cleans tracking
   git fetch --prune
4. git branch --merged to list merged
   git branch --merged

EXAMPLE diagram:



TIP: Automate cleanup with git fetch -p